

The task

Build an LLM agent that produces calibrated probabilities for questions about the future, the kind listed on prediction markets like Kalshi and Polymarket. Then run it.

We are not asking you to make forecasts we will hold you to. Nothing you produce here is a prediction we will trade on or score you against. We want to see the machinery, the reasoning behind it, and the reading that shaped it.

The question

Use this one:

How many seats will the Republican Party win in the U.S. House of Representatives in the 2026 midterm elections?

Everyone works on the same question so we can read submissions side by side. It was picked for three reasons:

- It has not happened. The election is on 3 November 2026, so there is no answer for you to look up and none for the model to remember.
- The prediction markets carry it, including seat-count distributions.
- It is not a yes or no question. How you represent an answer is a design decision, and we would like to see you make it deliberately rather than by default.

No political background is needed or expected, and nor is any finance background. This is a stripped-down version of the internship project, which forecasts a specific corporate event against a specific professional baseline. We kept the domain generic on purpose.

What the agent has to do

We are interested in the scaffolding around the model, not the prompt inside it. Two requirements:

- The agent works out for itself what information it needs and goes and gets it. We are not handing you a document set. A system that only reasons over context we specified is not answering the question we are asking.
- Every run is traced and persisted: what the agent did, in what order, what came back, and how it got from that to an answer. We will read these.

How you structure the agent is your call. Tell us in spec.md what you chose, what you considered and rejected, and why. If the design came out of something you read, say so.

It does not have to be sophisticated. Something that runs, terminates, handles a failed tool call and writes down what it did is worth a lot more than an elaborate architecture that never ran.

On the search tool. You will need one. Tavily, Serper and Brave all have free tiers that take a few minutes to set up. If you would rather not sign up for anything, a scripted fetch of a handful of pages is fine, but say so in the README. Do not spend your afternoon on this part.

What to send back

A repository, or a zip of one. We want the documentation as markdown, roughly like this:

README.md	what this is, how to run it, what you would do next
AGENTS.md	the instructions you gave your coding agent

(or CLAUDE.md)	
spec.md	what the system does, see below
literature.md	the review, see below
src/	the code, rough is fine
runs/	raw traces from your runs

Use whatever names suit you. The point is that the documentation is written the way you would write it for a coding agent, because that is how you should be working.

spec.md

What the system does: interfaces, data contracts, how failures are handled, and what is in a first version and what is not. Enough that two people reading it would build the same thing, and enough that we could run yours and get the same behaviour.

We ask for a spec rather than an essay because we want to recreate and check what you did, not just read what you concluded. Where you make a claim, we should be able to trace it. Cite the paper. Name the model and version you actually used. Say what you observed, not what you expected.

literature.md

People have worked on this before you. Find out who, and read enough to know what is already established, what is still contested, and what has been tried that did not work.

Three to five pieces of work is plenty. For each one: what it found, and what it changed about how you built. A paper that changed none of your decisions does not belong in the file. We would rather see a short list you actually used than a long one you summarised.

On using AI tools

Use them, and use the agentic ones: Claude Code, Codex, Cursor, whatever you work in. We use them here, and an exercise that pretended we did not would test the wrong thing. Directing these tools well and judging what they give back is a big part of the actual work. That is why we want the markdown you wrote to steer them.

Two conditions. Tell us what you used and what for. A sentence in the README is enough and it will not count against you. And submit only what you can defend. We will spend twenty minutes walking through your repository and asking why you made the calls you made.

Effort and timing

About four – five hours. The code will be rough, that is fine.

A partial submission is fine and we mean it. If you run out of time, send what you have and add a paragraph to the README saying what you would have done next and why. We would rather see three hours of clear work with an honest note than four hours of padding. If you are cutting, cut scope before you cut the trace logging or the literature file. Those two are what we read most closely.

(For a model the agent can call while it runs: this is the piece that actually matters for the exercise, and it is separate from the editor. Google AI Studio gives you a Gemini API key free with no credit card.)